# Lab Session 06

***Examine Image Processing: Loading, Arithmetic and Logical Operations, and Edge Detection***

## MATLAB Image Processing

The first step in MATLAB image processing is to understand that a digital image is composed of a two or three dimensional matrix of pixels. Individual pixels contain a number or numbers representing what grayscale or color value is assigned to it. Color pictures generally contain three times as much data as grayscale pictures, depending on what color representation scheme is used. Therefore, color pictures take three times as much computational power to process. In this Lab, the method for conversion from color to grayscale will be demonstrated and most processing will be done on grayscale images.

### Loading Image

To import an image from any supported graphics image file format, in any of the supported bit depths, use the **imread** function. This example reads a true color image into the MATLAB® workspace as the variable RGB.

**RGB = imread('football.jpg');**

If the image is not in the MATLAB workspace, the following command can be used to read an image file:

**myImage = imread('File name with path');**

### Displaying Image

After reading image data using the above function, we can display images using the **imshow** function. This function simply takes the array storing the image values as its only parameter.

Syntax:

**imshow(<variable name>)**

Example:

**imshow(myImage);**

### Writing an Image

Sometimes an image must be saved so that it can be transferred to a disk or opened with another program. This can be accomplished using the **imwrite** function, which allows you to save an image as any file type supported by MATLAB (the same set supported by imread).

The M-file below loads a bitmap file, converts it to grayscale, displays both, and saves the grayscale result as a JPEG. Notice how the JPEG image created is saved into the work folder when the M-file runs:

```
B = imread('image1.bmp');
figure(1), imshow(B);
C = rgb2gray(B);
figure(2), imshow(C);
imwrite(C, 'C:\MATLAB\workspace\new.jpg', 'jpg');
```

### Knowing the Size of an Image in Pixels

The size of the image in pixels can be found using the following command:

**[Rows, Cols] = size(myImage);**

## Image Resizing

Image resizing can be done using the following command:

**imresize(Image, [Parameters]);**

For example, if we read an image into the variable myImage using imread, we can resize it with:

**imresize(myImage, [256,256], 'nearest');**

This command converts an image of any size into a 256×256 image using the nearest-neighbor technique.

## Converting a Color Image into a Grayscale Image

A color image can be converted into a grayscale image using the function **rgb2gray**.

**myGrayImage = rgb2gray(myImage);**

## Converting a Color Image into a Binary Image

A color image can be converted into a binary image using the function **im2bw**:

**myBWImage = im2bw(myImage); % myImage is 2-D image data**

## Separating a Color Image into Three R, G, B Planes

If we read a color image using the imread() function, we get a 3-D matrix. To separate out the R, G and B planes, we can read each plane individually:

```
myImage = imread('RGBBar.bmp');
RED   = myImage(:,:,1);
GREEN = myImage(:,:,2);
BLUE  = myImage(:,:,3);
```

## Combining Three R, G, B Planes into a Color Image

If we have three R, G and B planes separately and want to create a color image from them, we can use the concatenate function:

**newImage = cat(3, RED, GREEN, BLUE);**

# Image Arithmetic Operations

Arithmetic operations such as image addition, subtraction, multiplication and division are possible. If there are two images I1 and I2, then the addition of the images is given by:

$$\mathbf{I(x,y) = I1(x,y) + I2(x,y)}$$

Where I(x,y) is the resultant image due to the addition of two images, and x and y are the coordinates of the image. Image addition is pixel-to-pixel. The value of a pixel should not cross the maximum value, which is 255 for an 8-bit greyscale image — when it exceeds 255, it should be clipped to 255.

To increase the overall brightness of an image, a constant value is added, depending on the brightness required. In the example program below, a constant value of 50 is added to the image.

To decrease brightness, a constant value is subtracted — in the example, 100 is subtracted from every pixel. Care should be taken that the pixel value does not go below 0. The subtract operation can also be used to obtain the complement (negative) image, in which every pixel is subtracted from the maximum value (255 for 8-bit greyscale).

Multiplication can be used to mask an image to obtain a region of interest: a black-and-white mask (binary image) containing 1s and 0s is multiplied with the image to get the resultant image (the function im2bw() can be used to obtain the binary mask). Multiplication can also be used to increase brightness.

Division results in floating-point numbers, so the data type required is floating point; it can be converted to an integer type when storing the image. Division can be used to decrease brightness, and can also be used to detect change between two images.

Execute the following MATLAB program:

```
clc
close all
I1 = imread('im.tif');
I2 = imread('im2.png');
subplot(2,2,1); imshow(I1); title('Original image I1');
subplot(2,2,2); imshow(I2); title('Original image I2');
I = I1 + I2;                    % Addition of two images
subplot(2,2,3); imshow(I); title('Addition of image I1+I2');
I = I1 - I2;                    % Subtraction of two images
subplot(2,2,4); imshow(I); title('Subtraction of image I1-I2');

figure;
subplot(2,2,1); imshow(I1); title('Original image I1');
I = I1 + 50;
subplot(2,2,2); imshow(I); title('Bright Image I');
I = I1 - 100;
subplot(2,2,3); imshow(I); title('Dark Image I');
M = imread('Mask.tif');
M = im2bw(M);                   % Converts into binary image having 0s and 1s
I = uint8(I1) .* uint8(M);      % Type casting before multiplication
subplot(2,2,4); imshow(I); title('Masked Image I');
```

# Logical Operations

Bitwise logical operations can be performed between the pixels of one or more images.

### AND / NAND

Used for:

- Computing the intersection of images
- Design of filter masks
- Slicing of grayscale images

### OR / NOR

Used for:

- Merging two images

### XOR / XNOR

Used for:

- Detecting a change in gray level in the image
- Checking similarity between two images

### NOT

Used for:

- Obtaining the negative image
- Making certain features clearer

Execute the following MATLAB program:

```
clc
close all
myimageA = imread('circle.jpg');
myimageA = rgb2gray(myimageA);
myimageB = imread('pentagon.jpg');
myimageB = rgb2gray(myimageB);

subplot(3,2,1)
imshow(myimageA), title('Image A');
% Display the Original Image B
subplot(3,2,2)
imshow(myimageB), title('Image B');
% Take a complement of Image A and Display it
subplot(3,2,3)
cimageA = ~myimageA;
imshow(cimageA), title('Complement of Image A');
% Take an Ex-OR of Image A and Image B and Display it
subplot(3,2,4)
xorimage = xor(myimageA, myimageB);
imshow(xorimage), title('Image A XOR Image B');
% Take OR of Image A and Image B and Display it
subplot(3,2,5)
orimage = myimageA | myimageB;
imshow(orimage), title('Image A OR Image B');
% Take AND of Image A and Image B and Display it
subplot(3,2,6)
andimage = myimageA & myimageB;
imshow(andimage), title('Image A AND Image B');
```

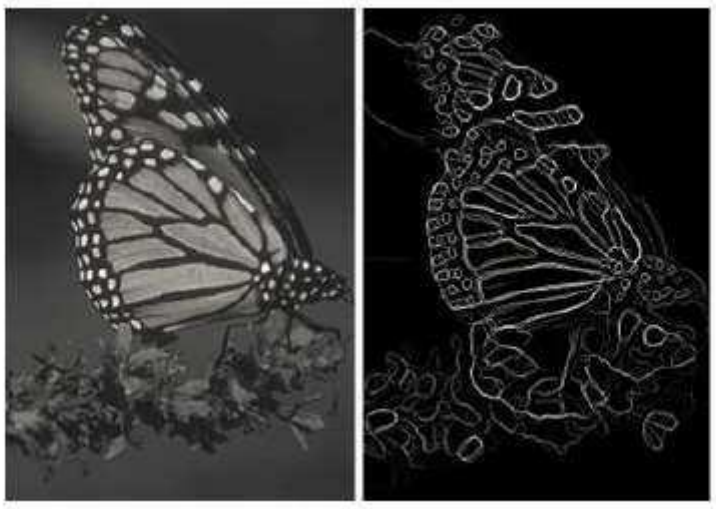
# Edge Detection

On modern computer systems, images are usually displayed on a screen or monitor with discrete image pixels, which create colors using different ratios of red, green, and blue (RGB). When computing with RGB, each pixel has a parameter specifying the level of each color, but for simplicity this lab deals only with black-and-white pixels (grayscale). An image is represented by storing the position and intensity of each pixel as a 2-D array, where each array element has an integer value between 0 and 255 (the range representable by an 8-bit number). The value 255 is white and 0 is black, with discrete shades of grey in between.

## Kernel Matrices

In image processing, many filter operations are applied to an image by performing a special operation called convolution with a matrix called a kernel. Kernels are typically 3×3 square matrices, although kernels of size 2×2, 4×4, and 5×5 are sometimes used. The values stored in the kernel directly relate to the results of applying the filter, and filters are characterized solely by their kernel matrix. For instance, the following kernels are used for detecting the vertical and horizontal edges in an image, and when applied result in the image shown below.

$$K_h = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \quad K_v = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$



*Original image and combined results of vertical and horizontal edge detection kernels*

## Image Filters

There are various types of filters and their applications, as well as real implementations using MATLAB, e.g. filters for image enhancement and edge extraction like the Sobel Edge Detector, and Gaussian and low-pass filters to reduce noise in signals and improve image quality.

## Sobel Edge Detector

Many applications in engineering and science require the correct identification of edges. For instance, many tools for automated manufacturing processes are equipped with cameras to detect markers (such as a thick black line) that might designate special reference points or physical locations.

The following are two such kernels for detecting horizontal and vertical edges, together called the Sobel operator. By inspection, the kernels are vertically and horizontally symmetric, and the sum of the kernel elements is zero.

This means that as the kernel passes over the image, vertically or horizontally aligned pixels that differ in intensity from their neighbors will be multiplied with the non-zero parts of the kernel, resulting in a non-zero pixel in the result. Since the sum of the kernel elements is zero, as soon as the kernel enters a region with uniform pixel values, the sum of the element-wise products becomes close to zero. Since 0 represents black, edges (nonzero) appear white and non-edges appear close to black.

We first read the source image using imread(), convert it to grayscale using rgb2gray(), define the Sobel matrices, and use the built-in convolution function conv2() to convolve the source image with the Sobel kernels:

```
function Sobel(im)
  % Read in the image and convert to gray
  orig = imread(im);
  grayscale = rgb2gray(orig);
  % Display the original and gray image
  figure(1); imshow(grayscale);
  figure(2); imshow(orig);
  % Define the Sobel kernels
  k_v = [-1 0 1; -2 0 2; -1 0 1];
  k_h = [1 2 1; 0 0 0; -1 -2 -1];
  % Convolve the gray image with Sobel kernels, store result in M1 and M2
  M1 = conv2(double(grayscale), double(k_v));
  M2 = conv2(double(grayscale), double(k_h));
  % Display the horizontal and vertical edges separately
  figure(3); imshow(abs(M1), []);
  figure(4); imshow(abs(M2), []);
  % Display the normalized vertical and horizontal edges combined
  figure(5); imshow((M1.^2 + M2.^2).^0.5, []);
end

% Call the Sobel() function with the image path as the argument.
Sobel('image-name.jpg');
```

## Noise Reduction and Gaussian Filters

In addition to feature extraction, filters can be used to denoise signals and images. Many filters can achieve this, and the optimal choice often depends on the application's requirements. One such filter is the Gaussian filter, named because its kernel is a discrete approximation of the Gaussian (normal) distribution.

While we could manually define the Gaussian equation, MATLAB provides the convenient function **fspecial()**, which accepts arguments for the distribution type and its parameters. We store the output of fspecial() in a variable, which we can convolve with the source image using conv2(). We define the filtering operation as a function so it can be applied to multiple images. MATLAB's standard library also includes **imnoise()**, used here to add noise to the test image.

```
function Gaussian(im, size, sigma, noiseType)
  % Display the original and gray image
  original = imread(im);
  grayscale = rgb2gray(original);
  figure(1); imshow(original);
  figure(2); imshow(grayscale);
  % Add noise to the grayscale image and display
  noisyImage = imnoise(grayscale, noiseType);
  figure(4); imshow(noisyImage);
  % Generate Gaussian matrix
  h = fspecial('gaussian', size, sigma);
  % Convolve the noised image with the Gaussian kernel
  M = conv2(double(grayscale), double(h));
  % Display the result
  figure(3); imshow((M.^2).^0.5, []);
end
```

# EXERCISE

Write and execute MATLAB programs for the following tasks. Paste your code and resulting output images for each task.

**Task # 1:** Read the color image 'RGBBar.bmp' given in the working directory and convert it into a Grey-scale image.

**Task # 2:** Read the color image 'RGBBar.bmp' given in the working directory. Find the size of the image, then resize it to 256×256.

**Task # 3:** Read any color image and separate it into its R, G and B planes. Display each plane individually using subplot. Then recombine the three planes using cat() to reconstruct the original image, and verify that the reconstructed image matches the original.

**Task # 4:** Prepare any two images of size 256×256 in Paint. Save them in JPEG format at 256 gray levels. Perform logical NOR and NAND operations between the two images. Write the program and paste your results.

**Task # 5:** Using image arithmetic, extract a region of interest from a grayscale image by multiplying it with a binary mask. Separately, take two images of the same scene captured at different times and use image division (or subtraction) to detect the change between them. Display and comment on both results.

**Task # 6:** Apply at least one other type of edge detection filter (e.g. Prewitt, Roberts, or Laplacian) and one image enhancement filter (e.g. median filter) on different images, and show their outputs. Compare your results with the Sobel detector used earlier in this lab.